

OFPPT • CMC Rabat

Wave Inc. • Témara

Rapport de stage de fin de formation

Présenté en vue de l'obtention du diplôme de

TECHNICIEN SPÉCIALISÉ EN DÉVELOPPEMENT DIGITAL

WAVE

Plateforme de Gestion Clients et Licences

Développement d'une application web full-stack
pour la gestion commerciale et la gestion des licences logicielles

Réalisé par

M. LEMCHAOURI Zakaria

Organisme : Wave Inc., Témara, Maroc

Encadrant professionnel : M. Mohamed Aherdane

Encadrante pédagogique : Mme. Ourrahte Malak

Soutenu le _____ / _____ / 2026 Devant le jury :

M./Mme. _____ Formateur Développement Digital **Président du jury**

M./Mme. _____ Formateur Développement Digital **Examineur**

Dédicaces

À ceux qui ont toujours cru en moi. . .

À mes parents,

Il y a des soutiens qu'on ne mesure qu'avec le recul. Chaque sacrifice que vous avez consenti, chaque mot d'encouragement glissé au bon moment, chaque regard plein de confiance ont été le carburant silencieux de ce parcours. Ce diplôme porte votre nom autant que le mien, et je vous en suis infiniment reconnaissant.

À ma famille,

Votre présence chaleureuse et votre soutien constant ont été un ancrage précieux, aussi bien dans les moments de doute que dans ceux de fierté. Merci d'avoir toujours été là.

À mes amis et camarades de promotion,

Ces deux années n'auraient pas eu la même saveur sans vous. Les sessions de travail tardives, les débats techniques qui s'enflammaient pour un rien, les fous rires qui décompressaient les journées intenses : tout cela a fait partie de la formation, au même titre que les cours. Merci pour tout.

À toutes les personnes qui croient que l'effort finit toujours par payer.

Avec toute ma gratitude et mon respect.

Remerciements

Ce rapport marque la fin d'une période de formation dense et particulièrement enrichissante. Avant d'en présenter le contenu, je tiens à exprimer ma reconnaissance envers toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Mes premiers remerciements vont naturellement à **Wave Inc.** pour m'avoir accueilli au sein de son équipe dans les meilleures conditions. Travailler au sein d'une agence digitale active sur des projets réels m'a permis de confronter rapidement mes acquis théoriques à des problématiques concrètes, et d'apprendre bien plus vite qu'aucun cours n'aurait pu me l'offrir seul.

Je remercie tout particulièrement **M. Mohamed Aherdane**, mon encadrant professionnel, pour sa disponibilité, la clarté de ses conseils et la confiance qu'il m'a accordée dans mes choix techniques. Ses retours m'ont systématiquement poussé à remettre en question mes solutions et à produire un travail plus rigoureux. J'ai beaucoup appris à son contact.

J'adresse également ma gratitude à **Mme. Ourrahte Malak**, encadrante pédagogique à l'OFPPT CMC Rabat, pour son suivi attentif tout au long de la formation et pour les repères méthodologiques qu'elle a su transmettre avec bienveillance. Sa disponibilité a été un vrai appui dans les moments de flottement.

Je remercie l'ensemble du corps formateur de l'OFPPT CMC Rabat pour la qualité de l'enseignement dispensé et pour avoir su nous transmettre, au-delà des compétences techniques, une vraie façon de penser le développement logiciel de manière professionnelle.

Enfin, je remercie les membres du jury pour le temps qu'ils consacreront à l'évaluation de ce travail.

Zakaria Lemchaouri

Table des matières

Dédicaces	
Remerciements	
Table des figures	iii
Liste des tableaux	iv
Acronymes	v
Introduction Générale	1
1 Contexte General du Projet	3
1.1 Introduction	4
1.2 Presentation de l'organisme d'accueil	4
1.2.1 Identite et historique	4
1.2.2 Implantation geographique	4
1.2.3 Positionnement et services	5
1.2.4 Methodologie de travail	6
1.2.5 Quelques realisations notables	6
1.3 Cadre du projet	6
1.3.1 Problematique	6
1.3.2 Analyse de l'existant	7
1.3.3 Solution proposee	8
1.3.4 Comparaison entre l'ancien systeme et WAVE	8
1.4 Analyse et specification des besoins	9
1.4.1 Besoins fonctionnels	9
1.4.2 Besoins non fonctionnels	12
1.5 Gestion du projet	13
1.5.1 Planification previsionnelle	13
1.6 Conclusion	14
2 Analyse et Conception	15

2.1	Introduction	16
2.2	Methodes et outils de modelisation	16
2.2.1	Langage de modelisation UML	16
2.2.2	Identification des acteurs	16
2.3	Diagrammes de cas d'utilisation	17
2.3.1	Diagramme de cas d'utilisation – Scheduler, Gestion Clients et Gestion Factures	17
2.3.2	Diagramme de cas d'utilisation – Manager	18
2.3.3	Diagramme de cas d'utilisation – Admin et Client	20
2.4	Diagrammes de sequence	23
2.4.1	Sequence 1 – Authentification et redirection par role	23
2.4.2	Sequence 2 – Commande et paiement PayPal	25
2.4.3	Sequence 3 – Verification automatique des licences	27
2.5	Diagramme de classes	29
2.6	Architecture technique	32
2.6.1	Architecture API-first	32
2.6.2	Gestion des roles et des acces	32
2.6.3	Gestion asynchrone et taches planifiees	33
2.7	Conclusion	33

Table des figures

1.1	Diagramme de Gantt previsionnel du projet (4 semaines)	13
2.1	Diagramme de cas d'utilisation – Scheduler, Clients et Factures	18
2.2	Diagramme de cas d'utilisation – Manager	20
2.3	Diagramme de cas d'utilisation – Admin et Client	22
2.4	Diagramme de sequence – Authentification et redirection par role	24
2.5	Diagramme de sequence – Commande et paiement PayPal	26
2.6	Diagramme de sequence – Verification automatique des licences	28
2.7	Diagramme de classes de l'application WAVE	30

Liste des tableaux

1.1	Donnees d'identification de Wave Inc.	5
1.2	Stack technologique de Wave Inc.	5
1.3	Analyse des limites du systeme existant	8
1.4	Comparaison entre l'existant et la solution WAVE	9
2.1	Principales relations Eloquent du systeme	32

Acronymes

Sigle	Signification
API	Application Programming Interface
REST	Representational State Transfer
RBAC	Role-Based Access Control
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSX	JavaScript XML
PHP	Hypertext Preprocessor
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
URL	Uniform Resource Locator
UI	User Interface
UX	User Experience
BD	Base de Données
SPA	Single Page Application
SMTP	Simple Mail Transfer Protocol
MVC	Modèle, Vue, Contrôleur
UML	Unified Modeling Language
CORS	Cross-Origin Resource Sharing
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
MAD	Dirham Marocain

Sigle	Signification
CRM	Customer Relationship Management
PDF	Portable Document Format
CSV	Comma-Separated Values
KPI	Key Performance Indicator
JWT	JSON Web Token
SDK	Software Development Kit
CI/CD	Continuous Integration et Continuous Deployment
FK	Foreign Key (Clé étrangère)
PK	Primary Key (Clé primaire)
MRR	Monthly Recurring Revenue (Revenu mensuel récurrent)
OFPPT	Office de la Formation Professionnelle et de la Promotion du Travail
CMC	Cité des Métiers et des Compétences

Introduction Générale

La transformation numérique occupe aujourd'hui une place centrale dans la stratégie des entreprises marocaines, tous secteurs confondus. Face à une concurrence de plus en plus vive et à des clients dont les attentes ne cessent de croître, les agences digitales et les éditeurs de logiciels ont besoin d'outils internes solides pour piloter leur activité, gérer leurs portefeuilles clients et assurer un suivi fiable de leurs licences logicielles. Dans la réalité, beaucoup de structures de taille intermédiaire s'appuient encore sur des solutions fragmentées : tableurs, échanges par email, fichiers partagés en vrac. Le résultat, c'est des pertes d'information régulières, des doublons qui s'accumulent et une visibilité insuffisante sur la santé réelle de l'activité.

C'est précisément ce constat qui est à l'origine de ce stage, réalisé au sein de **Wave Inc.**, une agence digitale établie à Témara. L'entreprise accompagne des clients marocains dans leur transition numérique en leur proposant des solutions web sur mesure ainsi que des produits SaaS. Avant le lancement de ce projet, la gestion des clients, des paiements, des licences et des contrats reposait en grande partie sur des processus manuels peu coordonnés, ce qui rendait le suivi quotidien laborieux et compliquait toute lecture globale de la performance commerciale.

La mission confiée consistait donc à concevoir et développer **WAVE**, une application web full-stack dédiée à la gestion commerciale et à la gestion de licences. L'idée centrale était simple : regrouper dans une seule plateforme cohérente l'ensemble des opérations du back-office, à savoir la gestion des clients, des paiements, des contrats, des factures et du pipeline commercial. En parallèle, les clients devaient disposer d'un espace personnel pour consulter leurs licences actives, leurs factures, leurs contrats, et passer des commandes directement en ligne via une intégration **PayPal**.

L'application a été conçue selon une architecture **API-first**, avec un backend en **Laravel 13** exposant une API REST sécurisée grâce à **Laravel Sanctum**, et un frontend en **React 19** construit avec **Vite**. Cette séparation claire entre les deux couches facilite la maintenance du code, améliore la scalabilité et offre une expérience utilisateur fluide sous forme de Single Page Application. Le système gère trois profils distincts : **admin**, **manager** et **client**, dont les droits d'accès sont contrôlés de façon granulaire

via un middleware de rôle côté serveur et un système de routes protégées côté client.

Parmi les fonctionnalités les plus notables, on peut citer la génération automatique de factures numérotées au format FAC-YYYY-NNNN, l'export de documents en PDF via **Laravel DomPDF**, un système de relances et d'alertes pour les contrats arrivant à expiration, un journal d'audit basé sur **Spatie ActivityLog**, et un tableau de bord analytique avec visualisation des revenus mensuels grâce à **Chart.js**. Des tâches planifiées envoient par ailleurs des emails automatiques aux clients dont les licences approchent de leur terme ou qui accusent un retard de paiement.

Ce rapport retrace l'ensemble de la démarche suivie pour mener ce projet à bien, en trois chapitres :

- **Chapitre 1 (Contexte Général du Projet)** : présentation de l'organisme d'accueil Wave Inc., analyse de la situation existante, identification des problèmes rencontrés et spécification des besoins fonctionnels et non fonctionnels de la solution.
- **Chapitre 2 (Analyse et Conception)** : modélisation du système à l'aide des outils UML (diagrammes de cas d'utilisation, de séquence et de classes), conception du schéma de base de données et définition de l'architecture technique retenue.
- **Chapitre 3 (Réalisation)** : présentation des choix technologiques justifiés, description des modules développés, aperçu des interfaces réalisées et bilan des fonctionnalités livrées.

À titre personnel, ce stage a représenté une expérience particulièrement marquante. Au-delà des compétences techniques approfondies, que ce soit en développement backend Laravel, en conception d'API REST, en intégration de paiement en ligne, en automatisation par tâches planifiées ou en gestion des permissions par rôle, c'est surtout la confrontation à un projet réel mené de bout en bout qui aura été la vraie valeur ajoutée. Ce rapport conclut deux années de formation en Développement Digital à l'OFPPT CMC Rabat et constitue une première immersion concrète dans les exigences du terrain.

CHAPITRE 1

Contexte General du Projet

1.1 Introduction

Ce premier chapitre pose les bases du projet en présentant l'environnement dans lequel il s'inscrit. On y trouve la description de l'organisme d'accueil, l'analyse de la situation avant le projet, les problèmes concrets qui ont motivé le développement de **WAVE**, ainsi que la spécification détaillée des besoins fonctionnels et non fonctionnels de la solution. L'objectif est de donner au lecteur une vision claire du "pourquoi" avant d'aborder le "comment" dans les chapitres suivants.

1.2 Présentation de l'organisme d'accueil

1.2.1 Identité et historique

Wave Inc. est une agence digitale à service complet, fondée le 26 mai 2015 au Maroc. L'entreprise est enregistrée sous forme juridique de Société à Responsabilité Limitée (SARL) avec un capital social de 500 000 DHS. Sa gouvernance repose sur un dirigeant principal associé à trois co-associés.

Depuis sa création, l'agence a connu plusieurs étapes de restructuration pour accompagner sa croissance. Un premier transfert de siège social accompagne d'une cession de parts sociales a eu lieu en novembre 2016, suivi d'une augmentation de capital en mars 2021 et d'un second transfert de siège en juillet 2022, ce qui témoigne d'une trajectoire de développement continue depuis dix ans.

1.2.2 Implantation géographique

Bien que la communication institutionnelle de l'agence mette en avant un ancrage stratégique au cœur de Rabat, son établissement principal est situé sur l'Avenue Mohammed V à Temara, à une quinzaine de minutes de la capitale. Cette localisation offre un double avantage : des coûts d'exploitation immobilière mieux maîtrisés, tout en maintenant une proximité directe avec les grands donneurs d'ordres institutionnels et les ministères basés à Rabat.

Pour renforcer son positionnement à l'international et capter des contrats de sous-traitance sur le marché européen, Wave Inc. dispose également d'un bureau de liaison commerciale en France, situé au Blanc Mesnil en région parisienne.

Le tableau 1.1 récapitule les principales données administratives et de contact de l'agence.

Table 1.1: Donnees d'identification de Wave Inc.

Parametre	Donnee
Denomination sociale	WAVE INC. SARL
Date de creation	26 mai 2015
Forme juridique	SARL – Capital social : 500 000 DHS
Siege social	5, Lot Mohamed Et Jamila, 1er Etage, Temara
Etablissement principal	5, Avenue Mohammed V, 12000 Temara, Maroc
Bureau secondaire	10, Avenue de Rome, 93150 Le Blanc Mesnil, France
Horaires	Lundi au vendredi, 09h00 – 18h00
Telephone	+212 (0) 537 612 736 / +212 (0) 809 893 112
Email general	contact@wave.ma
Email RH	rh@wave.ma

1.2.3 Positionnement et services

Wave Inc. se positionne comme un partenaire technologique de bout en bout pour ses clients, en couvrant l'ensemble du cycle de vie d'un projet numerique. Cette approche globale se distingue des agences specialisees sur un seul metier comme le design ou le referencement.

Les services proposes couvrent le developpement web et mobile, la conception d'interfaces utilisateur, l'integration de systemes via des API, le deploiement et la maintenance d'applications, ainsi que l'accompagnement dans la transformation digitale des entreprises.

Sur le plan technique, l'agence maitrise un ecosysteme technologique large et coherent, detaille dans le tableau 1.2.

Table 1.2: Stack technologique de Wave Inc.

Couche	Technologies utilisees
Frontend	React JS, Vue.js, Angular, Nuxt.js, TypeScript
Backend	Laravel, Symfony, CodeIgniter, CakePHP
Mobile	React Native, Flutter, Ionic
CMS	WordPress, Drupal, Prestashop, WooCommerce
APIs	REST, SOAP, WebSockets

1.2.4 Methodologie de travail

La methodologie d'ingenierie appliquee par Wave Inc. s'inspire des frameworks agiles. Les projets sont decoupees en cycles de developpement courts et iteratifs, ce qui permet une mise en production progressive et un ajustement continu en fonction des retours clients. Les problematiques de performance, de securite et de referencement sont integrees des la phase de prototypage, plutot que traitees tardivement comme de simples correctifs.

L'onboarding d'un nouveau projet suit trois jalons : une phase de proposition avec estimation de l'effort sous 24 heures, une phase de cadrage technique avec le client, puis un demarrage immediat apres signature du contrat.

Concernant les modeles contractuels, l'agence propose deux formules. Le forfait global est utilise quand le perimetre fonctionnel est bien defini en amont, avec un prix ferme et un pilotage entierement pris en charge par l'agence. Le mode regie est prefere pour les projets de longue duree ou les besoins evoluent, avec une equipe dediee dont la composition peut etre ajustee selon la charge.

1.2.5 Quelques realisations notables

La credibilite technique de Wave Inc. est illustree par des projets realises pour des structures publiques et privees exigeantes.

L'agence a assure le developpement du site institutionnel officiel du Conseil de la Concurrence du Royaume du Maroc, une autorite administrative independante de rang constitutionnel. Ce type de projet impose des criteres stricts en matiere d'accessibilite, de securite des donnees et de disponibilite de l'infrastructure pour faire face aux pics de trafic lors de publications de rapports sensibles.

Elle a egalement concu la plateforme numerique complete de 2S Africa Travel, une agence de voyage haut de gamme agreee par le Ministere du Tourisme marocain. La solution integre des modules de tarification dynamique, la gestion de flux de reservations complexes et des connexions de paiement securisees avec des operateurs comme Visa, Mastercard et Payzone.

Ces references attestent de la capacite de l'agence a livrer des solutions logicielles robustes dans des contextes ou la fiabilite et la securite ne sont pas negociables.

1.3 Cadre du projet

1.3.1 Problematique

Malgre son expertise technique et la qualite de ses realisations pour ses clients, Wave Inc. n'avait pas consacre le meme niveau d'attention a ses propres outils de ges-

tion interne. La gestion du portefeuille clients, des licences logicielles, des paiements et des contrats reposait sur des fichiers Excel et Google Sheets mis à jour manuellement. Chaque information existait dans un fichier différent, sans lien entre les données, et sans aucun mécanisme d'alerte automatique.

Cette situation engendrait plusieurs problèmes concrets au quotidien. Un manager ne savait pas, sans aller fouiller dans plusieurs fichiers, si un client avait des licences sur le point d'expirer ou des paiements en retard. La création d'une facture nécessitait de recopier manuellement les informations du client depuis une feuille vers une autre. Il n'existait aucune trace structurée des échanges commerciaux avec un client, aucun historique consolidé des paiements et aucune vision globale des performances mensuelles sans construire soi-même des tableaux récapitulatifs.

Plus l'activité grandissait, plus cette approche montrait ses limites. Le risque d'erreur augmentait, le temps consacré aux tâches administratives répétitives s'alourdissait et la qualité du suivi client se dégradait faute d'un outil centralisé.

1.3.2 Analyse de l'existant

Avant le développement de **WAVE**, le processus de gestion d'un client ressemblait à ceci : les informations de base étaient saisies dans un fichier Excel partagé sur Google Drive, les paiements recus étaient notés dans un autre onglet, les licences attribuées étaient listées dans un troisième tableau, et les rappels d'échéances de contrats étaient gérés manuellement par le manager concerné, souvent via des alertes dans son calendrier personnel.

Cette fragmentation posait plusieurs problèmes identifiés lors de l'analyse de l'existant.

Premièrement, il n'y avait aucune source de vérité unique. Une information modifiée dans un fichier n'était pas automatiquement repercute ailleurs, ce qui générait des incohérences entre les données.

Deuxièmement, le suivi des licences était entièrement manuel. Aucun système n'informait automatiquement l'équipe qu'une licence allait expirer dans une semaine. C'est le manager qui devait consulter régulièrement le fichier et faire le calcul lui-même.

Troisièmement, la génération de factures était une opération longue et répétitive. Il fallait ouvrir un modèle Word ou Excel, recopier les informations du client, calculer les montants, exporter en PDF et envoyer par email. Aucune numérotation automatique n'existait, ce qui compliquait la traçabilité des documents.

Quatrièmement, les clients n'avaient aucune visibilité sur leur propre situation. Ils ne pouvaient pas consulter leurs licences actives, leurs factures ou leurs contrats sans contacter directement leur manager, ce qui générait une charge supplémentaire pour

l'équipe.

Le tableau 1.3 synthétise les principales limites identifiées.

Table 1.3: Analyse des limites du système existant

Domaine	Probleme identifié	Impact
Données clients	Informations dispersées dans plusieurs fichiers sans synchronisation	Élevé
Licences	Aucune alerte automatique d'expiration	Élevé
Paiements	Suivi manuel, pas d'historique consolidé	Moyen
Facturation	Génération manuelle, pas de numérotation auto	Moyen
Contrats	Pas d'alerte sur les échéances proches	Élevé
Espace client	Aucun accès autonome pour les clients	Moyen
Reporting	Aucun tableau de bord, construction manuelle	Élevé
Traçabilité	Aucun journal des actions effectuées	Moyen

1.3.3 Solution proposée

Face à ces constats, il a été décidé de développer **WAVE**, une application web full-stack centralisée qui regroupe l'ensemble de ces fonctionnalités dans une interface unique et cohérente.

La solution répond aux lacunes identifiées de la manière suivante. Toutes les données clients, paiements, licences, factures et contrats sont stockées dans une base de données relationnelle unique, ce qui élimine les incohérences entre fichiers. Des tâches planifiées envoient automatiquement des emails d'alerte aux clients dont les licences expirent dans sept jours ou qui accusent un retard de paiement. La facturation est entièrement automatisée avec une numérotation séquentielle au format FAC-YYYY-NNNN et une génération PDF en un clic. Les clients disposent d'un espace personnel où ils peuvent consulter leurs licences actives, leurs factures, leurs contrats et passer de nouvelles commandes via PayPal. Un journal d'audit basé sur Spatie ActivityLog trace toutes les actions effectuées sur les ressources sensibles. Un tableau de bord analytique avec visualisation graphique des revenus mensuels permet aux managers de piloter leur activité sans construire manuellement de tableaux récapitulatifs.

1.3.4 Comparaison entre l'ancien système et WAVE

Le tableau 1.4 présente une synthèse comparative entre la situation avant le projet et les apports de la nouvelle plateforme.

Table 1.4: Comparaison entre l'existant et la solution WAVE

Fonctionnalité	Avant WAVE	Avec WAVE
Gestion clients	Fichiers Excel separes	Base de donnees centralisee
Alertes licences	Manuelle ou inexistante	Email automatique J-7
Generation facture	Copier-coller manuel	PDF genere en un clic
Numerotation	Pas de standard	FAC-YYYY-NNNN auto
Espace client	Inexistant	Portail self-service
Paieement en ligne	Inexistant	Commande via PayPal
Tableau de bord	Construction manuelle	KPIs et graphiques temps reel
Journal d'audit	Inexistant	Traçabilité complete (Spatie)
Relances retards	Email manuel du manager	Envoi automatique planifie
Pipeline CRM	Aucun	Module opportunités integre

1.4 Analyse et specification des besoins

La specification des besoins a ete conduite apres plusieurs echanges avec l'encadrant professionnel M. Mohamed Aherdane, en analysant les processus existants et en identifiant les points de friction les plus critiques. Les besoins se repartissent en deux categories : fonctionnels, qui decrivent ce que le systeme doit faire, et non fonctionnels, qui decrivent comment il doit le faire.

1.4.1 Besoins fonctionnels

Les besoins fonctionnels ont ete organises par module, en distinguant les fonctionnalités selon le role de l'utilisateur concerne.

1. Authentification et gestion des roles

- Connexion securisee par email et mot de passe avec generation d'un token Bearer via Laravel Sanctum.
- Deconnexion avec invalidation immediate du token.
- Redirection automatique vers le tableau de bord correspondant au role apres connexion (admin, manager, client).
- Controle d'accès granulaire : chaque groupe de routes est protege par un middleware de verification du role.
- Provisionnement d'un compte utilisateur pour un client par le manager, sans que le client n'ait a s'inscrire lui-meme.

2. Gestion des clients

- Creation, modification et suppression douce (soft delete) d'une fiche client avec les champs nom, email, telephone, societe, statut et notes.
- Recherche et filtrage de la liste des clients par nom, email ou statut (actif, en attente, en retard).
- Export de la liste complete des clients au format CSV.
- Consultation d'un journal d'audit par client, trace par Spatie ActivityLog, listant toutes les actions effectuees sur sa fiche.

3. Gestion des paiements

- Enregistrement, modification et suppression douce de paiements, avec les informations montant (en MAD), date, reference et type (abonnement, licence, autre).
- Calcul automatique du total des paiements par client.
- Export du relevé de paiements d'un client au format PDF via Laravel DomPDF.

4. Gestion des licences

- Attribution d'une licence produit a un client avec date d'attribution et calcul automatique de la date d'expiration selon la duree du produit.
- Revocation d'une licence.
- Mise a jour automatique du statut de la licence : active, bientôt expiree (a moins de 7 jours) ou expiree.
- Envoi d'un email de rappel automatique au client dont la licence expire dans exactement 7 jours.

5. Gestion des factures

- Creation, modification et suppression douce de factures avec numerotation sequentielle automatique au format FAC-YYYY-NNNN.
- Telechargement individuel de chaque facture au format PDF.
- Liaison automatique entre une facture et le paiement correspondant.

6. Gestion des contrats

- Creation, modification et suppression de contrats avec dates de debut et de fin, montant en MAD et manager responsable.
- Mise a jour automatique du statut du contrat (actif ou expire) selon la date du jour.
- Alerte sur les contrats expirant dans les prochains jours, accessible depuis le tableau de bord.

7. Relances commerciales

- Creation et gestion de relances (follow-ups) associees a un client, avec note, date prevue et statut (faite ou non).
- Envoi automatique d'un email de rappel aux clients en statut "en retard" chaque lundi matin.

8. Pipeline commercial

- Creation, modification et suppression d'opportunités commerciales avec titre, description, montant estime en MAD, statut et date de cloture prevue.
- Attribution d'une opportunité a un client et a un manager responsable.

9. Tickets de support

- Soumission d'un ticket de support par le client depuis son espace personnel, avec sujet et message.
- Consultation et mise a jour du statut des tickets par le manager.

10. Tableau de bord et statistiques

- Affichage des KPIs principaux : nombre total de clients, clients payes, clients en attente, clients en retard et revenu mensuel recurrent (MRR).
- Graphique des revenus par mois sur l'annee selectionnee.
- Alerte visuelle si des licences expirent dans les 7 prochains jours.

11. Espace client

- Consultation des licences actives avec dates d'expiration.
- Consultation et telechargement des factures et contrats.
- Soumission et suivi de tickets de support.
- Consultation des notifications in-app et marquage comme lue.
- Acces a la boutique de produits et passation de commandes via PayPal.
- Export du releve personnel en PDF.

12. Administration

- Creation, consultation et suppression de comptes managers par l'admin.
- Creation, modification, suppression et activation/desactivation de produits avec prix en MAD et duree de licence en jours.
- Definition et mise a jour des objectifs mensuels par manager.

1.4.2 Besoins non fonctionnels

Au-delà des fonctionnalités, la solution devait répondre à un ensemble d'exigences de qualité.

1. Sécurité

Toutes les routes de l'API sont protégées par un token Bearer obligatoire. Tout accès sans token valide est immédiatement rejeté avec une erreur HTTP 401. Le middleware de rôle vérifie, en plus de l'authentification, que l'utilisateur connecté dispose bien des droits nécessaires pour accéder à la ressource demandée. Les mots de passe sont hachés avec bcrypt. Un intercepteur Axios côté frontend purge le token stocké en local et redirige vers la page de connexion en cas de réponse 401 du serveur. Les politiques CORS sont configurées pour n'accepter que les requêtes provenant du frontend autorisé.

2. Performance

Les emails sont envoyés de manière asynchrone via le système de queues de Laravel, de sorte que les actions de l'utilisateur ne soient jamais bloquées par un envoi d'email. Les listes de clients et de paiements sont paginées pour éviter de charger des volumes de données trop importants en une seule requête.

3. Ergonomie

L'interface doit être accessible sans formation particulière. La navigation doit être intuitive et les actions importantes doivent être accessibles en un minimum de clics. Chaque rôle dispose de son propre layout avec uniquement les menus qui le concernent, ce qui évite toute confusion entre les espaces.

4. Fiabilité

Toutes les entrées utilisateur sont validées côté serveur avant tout traitement. Les suppressions de clients, paiements et factures sont implémentées en soft delete pour éviter toute perte accidentelle de données. Les actions critiques comme la suppression d'un compte ou la modification d'un statut sont protégées par des règles métier strictes.

5. Maintenabilité

Le backend est organisé en modules indépendants par domaine fonctionnel. Le frontend est structuré en composants React réutilisables avec une séparation claire entre la logique d'état et la présentation. Cette organisation facilite l'ajout de nouveaux modules sans impacter les fonctionnalités existantes.

1.5 Gestion du projet

Le stage s'est deroule sur une duree de quatre semaines. La gestion du projet a suivi une approche MVP (Minimum Viable Product) : developper d'abord les fonctionnalites essentielles du systeme, a savoir l'authentification, la gestion des clients et des paiements, avant d'enrichir progressivement la plateforme avec les modules plus avances comme les notifications, le pipeline commercial et l'espace client.

Le travail a ete realise en autonomie, avec des points reguliers avec M. Mohamed Aherdane pour valider les choix techniques et les livrables de chaque phase. Le code source a ete versionne sur GitHub via Git tout au long du developpement, ce qui a permis de conserver un historique clair des modifications et de revenir a des etats anterieurs si necessaire.

1.5.1 Planification previsionnelle

La figure 1.1 presente la repartition temporelle des differentes phases du projet sous forme de diagramme de Gantt.

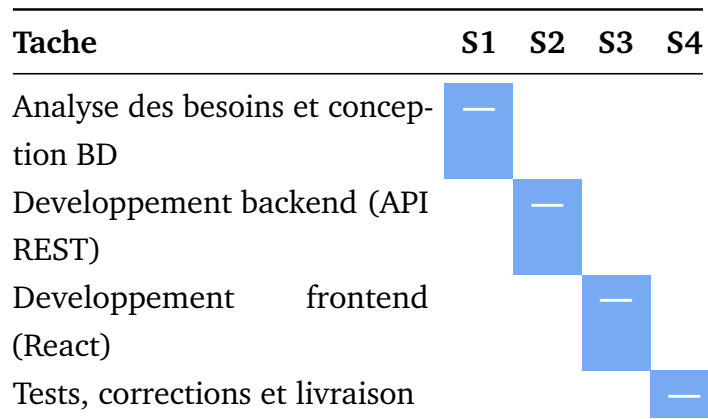


Figure 1.1: Diagramme de Gantt previsionnel du projet (4 semaines)

La premiere semaine a ete consacree a l'analyse des besoins, a l'etude de l'existant, a la conception de la base de donnees et au maquetage des interfaces principales. La deuxieme semaine a porte sur le developpement du backend : mise en place de l'API REST, authentification, gestion des clients, paiements, licences, factures et contrats. La troisieme semaine a ete dediee au frontend : construction des layouts par role, developpement des pages, integration des appels API et mise en place de l'espace client. La quatrieme semaine a ete reservee aux tests fonctionnels, corrections des bugs identifies et finalisation des livraisons.

1.6 Conclusion

Ce premier chapitre a permis de poser le cadre du projet. La presentation de Wave Inc. illustre une agence avec une expertise technique solide et des references serieuses, mais dont les outils de gestion interne n'etaient pas a la hauteur de cette expertise. L'analyse de l'existant a mis en evidence des lacunes significatives : donnees dispersees, aucune automatisisation des alertes, generation de documents manuelle et aucune visibilite offerte aux clients sur leur propre situation.

La solution **WAVE** repond directement a ces problemes en centralisant l'ensemble des processus dans une plateforme unique, structuree autour d'une architecture API-first moderne et d'un systeme de roles permettant a chaque utilisateur d'acceder uniquement a ce qui le concerne.

Le chapitre suivant sera consacre a la phase d'analyse et de conception, avec la modelisation UML du systeme, la conception de la base de donnees relationnelle et la definition de l'architecture technique retenue.

CHAPITRE 2

Analyse et Conception

2.1 Introduction

Après avoir posé le contexte du projet et spécifié les besoins dans le chapitre précédent, ce deuxième chapitre aborde la phase d'analyse et de conception. C'est une étape charnière : elle transforme des besoins exprimés en langage naturel en une architecture logicielle précise et documentée, sur laquelle le développement pourra s'appuyer directement.

Nous présentons ici les outils et méthodes de modélisation retenus, les acteurs du système et leurs interactions à travers les diagrammes de cas d'utilisation, les diagrammes de séquence illustrant les scénarios principaux, ainsi que le diagramme de classes qui constitue la base structurelle de la base de données de l'application.

2.2 Méthodes et outils de modélisation

2.2.1 Langage de modélisation UML

UML (Unified Modeling Language) est un langage de modélisation graphique standardisé, largement adopté dans le domaine du génie logiciel. Il permet de visualiser, spécifier et documenter les différentes dimensions d'un système, aussi bien sa structure statique que son comportement dynamique.

Dans le cadre de ce projet, UML a été utilisé pour produire trois types de diagrammes complémentaires :

- Les **diagrammes de cas d'utilisation**, qui décrivent les interactions entre les acteurs et le système, et permettent de valider que l'ensemble des besoins fonctionnels est bien couvert.
- Les **diagrammes de séquence**, qui retracent le déroulement chronologique des échanges entre les composants du système lors des scénarios les plus importants.
- Le **diagramme de classes**, qui représente la structure des données, les entités du domaine et leurs relations, et sert de référence directe pour la conception de la base de données.

2.2.2 Identification des acteurs

Un acteur est toute entité externe qui interagit avec le système. Pour l'application WAVE, quatre acteurs principaux ont été identifiés.

L'**Admin** est l'administrateur de la plateforme. Il dispose des accès les plus étendus : il peut gérer les comptes managers, le catalogue de produits, et définir les objectifs mensuels de chaque manager. Il hérite également de toutes les fonctionnalités du manager.

Le Manager est le commercial de l'agence. Il prend en charge le suivi quotidien des clients : gestion des paiements, attribution des licences, suivi des contrats et des relances, pilotage du pipeline commercial et consultation des tickets de support.

Le Client est l'utilisateur final de l'espace client. Il consulte ses licences actives, ses factures et ses contrats, soumet des tickets de support et peut passer des commandes directement en ligne via PayPal.

Le Scheduler (Cron) est un acteur systeme qui represente le planificateur de taches de Laravel. Il declenche automatiquement des traitements en arriere-plan : verification quotidienne des licences, envoi d'emails de rappel et generation du rapport mensuel.

2.3 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation ont ete organises par acteur afin de garder une lecture claire malgre le nombre important de fonctionnalites couverts par la plateforme. Trois diagrammes ont ete produits, chacun couvrant un perimetre fonctionnel distinct.

2.3.1 Diagramme de cas d'utilisation – Scheduler, Gestion Clients et Gestion Factures

Le premier diagramme (Figure 2.1) regroupe les fonctionnalites liees a la gestion des clients, a la facturation et aux automatisations declenchees par le planificateur de taches.

Cote gestion des clients, le manager peut lister, rechercher, creer, modifier et supprimer des fiches clients. Il peut egalement provisionner un compte utilisateur pour un client et exporter la liste au format CSV. La consultation de la fiche client sert de point d'entree pour acceder aux autres modules : paiements, licences, contrats et relances.

Cote facturation, le manager cree des factures avec numerotation automatique au format FAC-YYYY-NNNN, les modifie, les supprime et peut telecharger chaque facture en PDF.

Le Scheduler prend en charge trois automatisations : il verifie et met a jour le statut des licences chaque jour, envoie des rappels d'expiration sept jours avant l'echancee, envoie des rappels de retard de paiement chaque lundi, et genere un rapport mensuel envoye automatiquement aux admins.

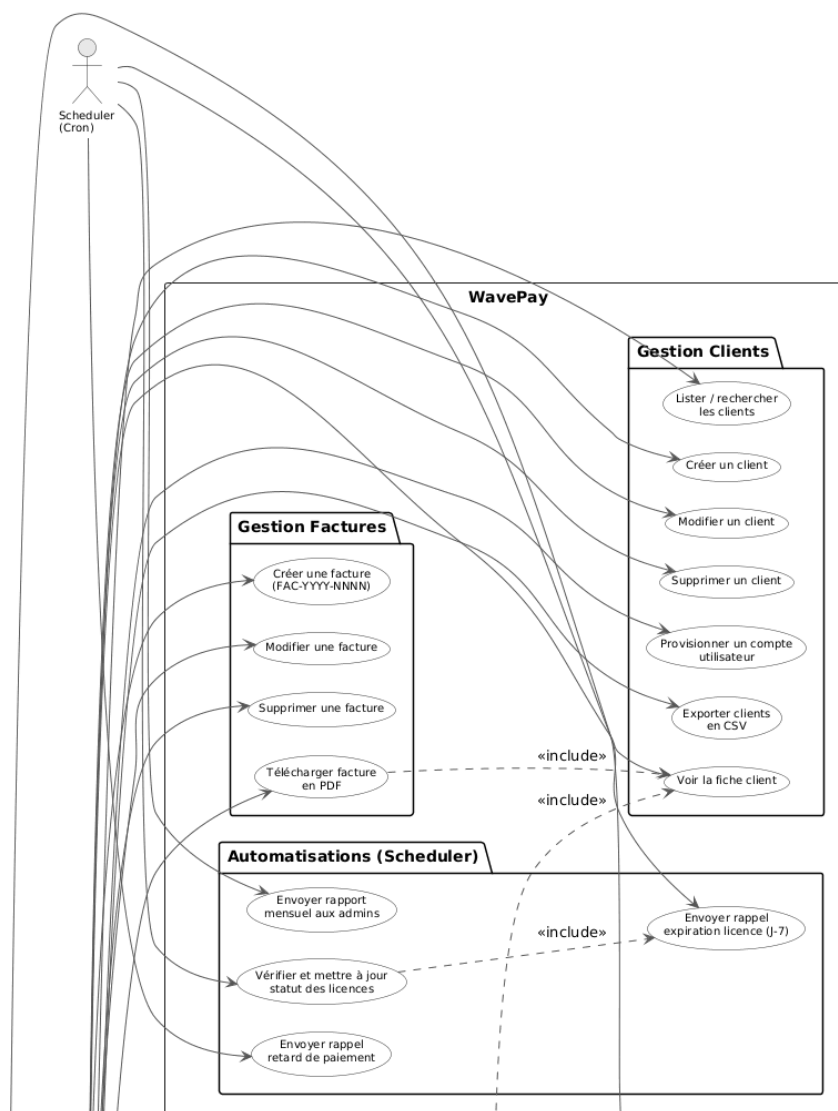


Figure 2.1: Diagramme de cas d'utilisation – Scheduler, Clients et Factures

2.3.2 Diagramme de cas d'utilisation – Manager

Le deuxième diagramme (Figure 2.2) est entièrement dédié au rôle Manager et couvre l'ensemble de son périmètre opérationnel.

La **gestion des contrats** permet de créer, modifier et supprimer des contrats, et de visualiser directement depuis le tableau de bord les contrats qui expirent bientôt.

Le module de **relances commerciales** (follow-ups) offre la possibilité de créer, modifier et supprimer des relances associées à un client, et de les marquer comme effectuées une fois réalisées.

Le **pipeline commercial** centralise la gestion des opportunités CRM : lister, créer, modifier et supprimer des opportunités avec leur montant estimé et leur date de clôture prévue.

La section **dashboard et reporting** donne accès au tableau de bord avec KPIs et

graphique des revenus mensuels, ainsi qu'au journal d'activité (audit log Spatie) par client.

Enfin, la **gestion des paiements** couvre l'enregistrement, la modification, la suppression et l'export en PDF du relevé de paiements, tandis que la **gestion des licences** permet d'attribuer, de révoquer et de consulter les licences d'un client.

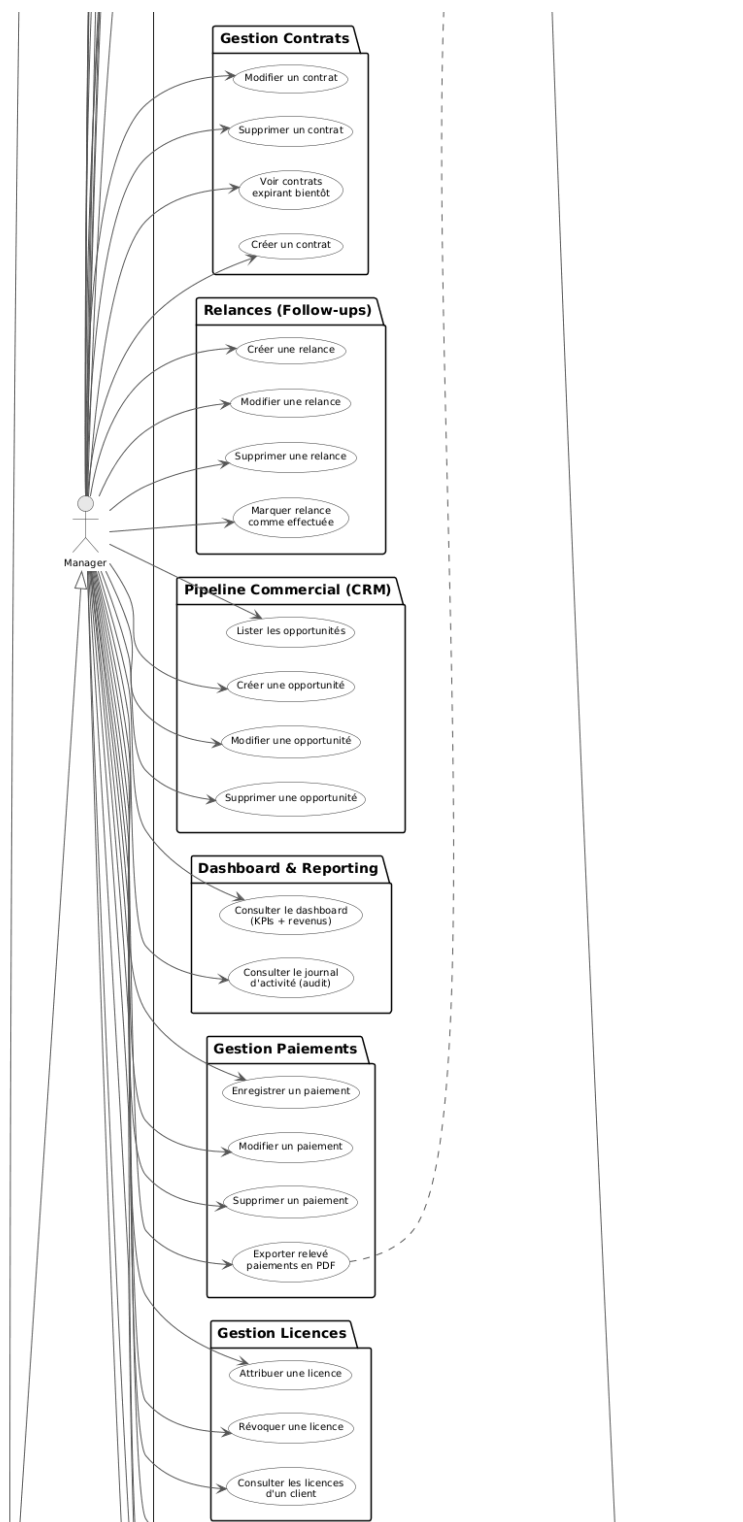


Figure 2.2: Diagramme de cas d'utilisation – Manager

2.3.3 Diagramme de cas d'utilisation – Admin et Client

Le troisième diagramme (Figure 2.3) regroupe les cas d'utilisation spécifiques à l'Admin et au Client.

Du côté **Admin**, le module d'authentification est commun à tous les rôles (connexion,

deconnexion, consultation du profil via /me). Le module de gestion admin lui permet de gérer les managers en CRUD complet, de gérer le catalogue produits, d'activer ou désactiver un produit et de définir les objectifs mensuels de chaque manager. Il partage également l'accès aux tickets de support en vue manager.

Du côté **Client**, l'espace personnel lui donne accès à son profil et à ses licences actives, à ses factures avec téléchargement PDF, à ses contrats et à ses notifications in-app avec possibilité de les marquer comme lues. Il peut exporter son relevé personnel en PDF. Le module boutique lui permet de parcourir le catalogue de produits actifs et de passer une commande, ce qui inclut la redirection vers PayPal et la confirmation du paiement via l'API PayPal SDK.

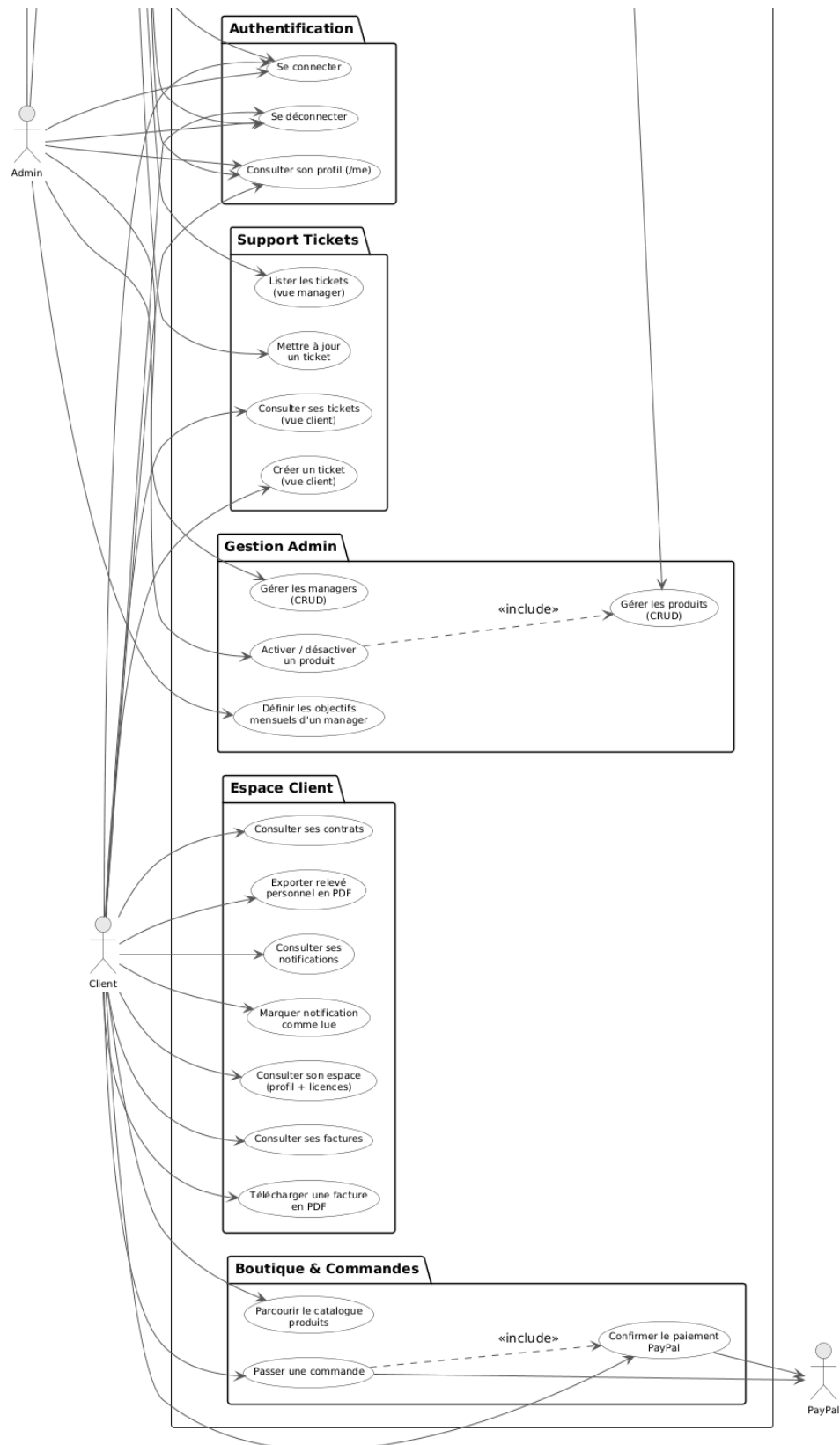


Figure 2.3: Diagramme de cas d'utilisation – Admin et Client

2.4 Diagrammes de sequence

Les diagrammes de sequence permettent de représenter dans l'ordre chronologique les échanges entre les différents composants du système pour un scénario donné. Trois scénarios ont été sélectionnés parmi les plus représentatifs et les plus complexes de l'application.

2.4.1 Sequence 1 – Authentification et redirection par rôle

Ce premier diagramme de sequence (Figure 2.4) couvre deux scénarios étroitement liés : la connexion d'un utilisateur et l'accès à une route protégée.

Lors de la connexion, l'utilisateur saisit son email et son mot de passe dans le formulaire React. Le frontend envoie une requête POST vers `/api/v1/login`. Le backend interroge la base de données pour vérifier l'existence du compte et la validité du mot de passe. En cas d'échec, une réponse 401 est retournée avec un message d'erreur affiché côté client. Si les identifiants sont corrects, un token Sanctum est généré et stocké dans la table `personal_access_tokens`. Le backend retourne ce token accompagné des informations de l'utilisateur (id, nom, rôle). Le frontend stocke le token dans le `localStorage` sous la clé `wave_token`, puis redirige automatiquement selon le rôle : vers `/admin/dashboard` pour un admin, vers `/manager/dashboard` pour un manager, ou vers `/client` pour un client.

Lorsque l'utilisateur navigue vers une route protégée, le frontend joint automatiquement le token dans l'en-tête HTTP `Authorization: Bearer`. Le middleware Sanctum vérifie la validité du token dans la table `personal_access_tokens`. Si le token est invalide ou expire, le backend retourne un 401, l'intercepteur Axios purge le `localStorage` et redirige vers la page de connexion. Si le token est valide, l'utilisateur est injecté dans la requête et l'accès est accordé.

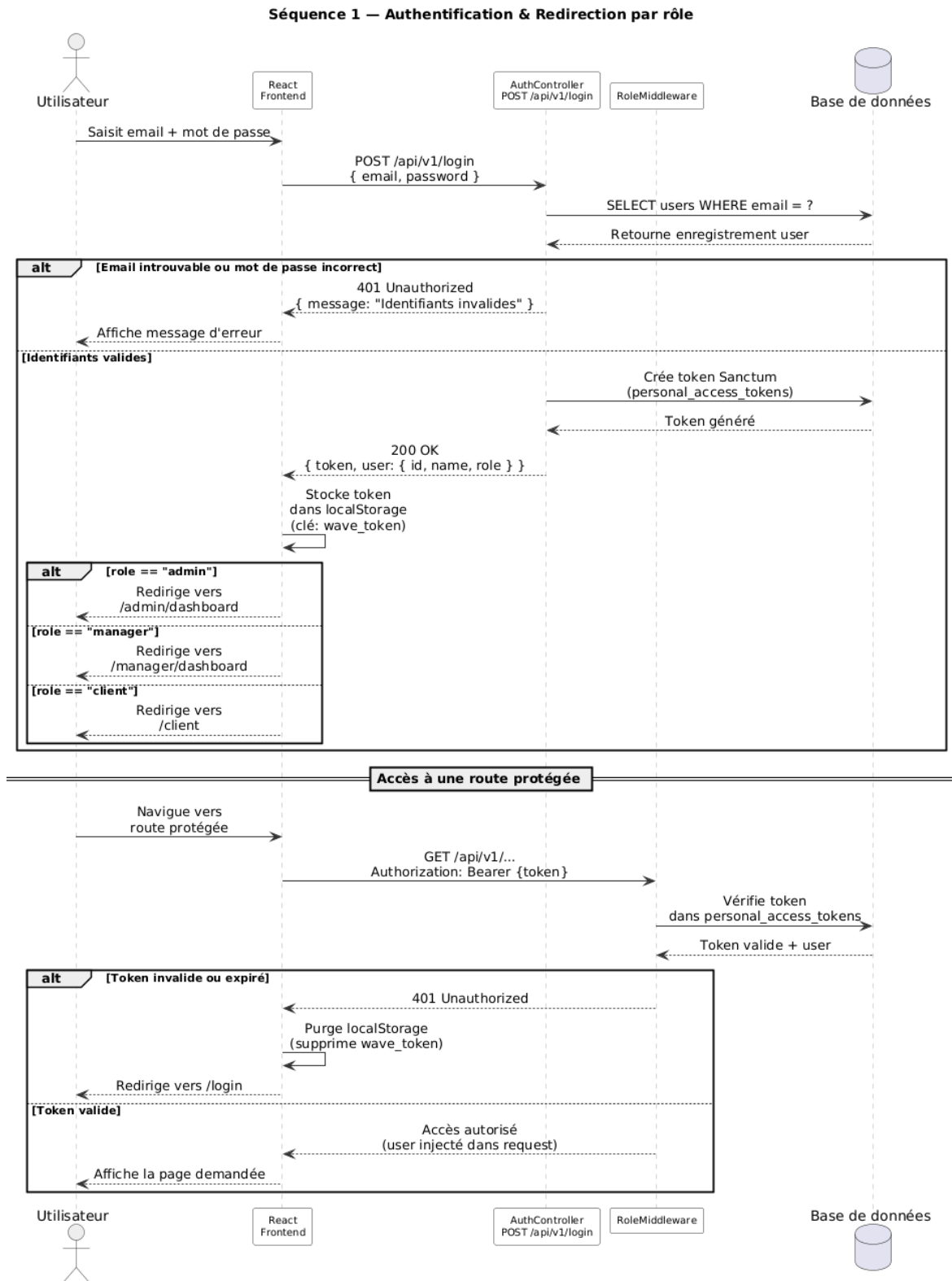


Figure 2.4: Diagramme de sequence – Authentification et redirection par rôle

2.4.2 Sequence 2 – Commande et paiement PayPal

Le deuxième diagramme de sequence (Figure 2.5) retrace le flux complet d'une commande depuis la boutique client jusqu'à l'attribution automatique de la licence après paiement confirmé.

Le client parcourt le catalogue en appelant `GET /api/v1/client/shop`, qui retourne les produits dont le champ `is_active` est vrai. Quand il clique sur "Commander", le frontend envoie une requête `POST` vers `/api/v1/client/shop/orders` avec l'identifiant du produit. Le backend vérifie que le produit est actif, récupère son prix, puis crée un ordre PayPal via le SDK `srmlive/paypal` avec le montant en MAD. L'identifiant et l'URL d'approbation PayPal sont retournés. Une commande est insérée en base avec le statut `pending` et l'identifiant PayPal. Le frontend redirige le client vers la page de paiement PayPal via l'URL d'approbation.

Après validation du paiement sur PayPal, le client est redirigé vers l'application. Le frontend envoie une requête `POST` vers `/api/v1/client/shop/orders/{order_id}/capture`. Le backend récupère la commande, appelle `captureOrder` sur le SDK PayPal avec l'identifiant PayPal. Si le paiement est refusé, le statut de la commande est mis à `cancelled` et une erreur 402 est retournée. Si la capture est confirmée, le backend met le statut à `completed`, crée un enregistrement de paiement avec type `license`, puis insère une entrée dans la table pivot `client_license` avec les dates d'attribution et d'expiration calculées automatiquement selon la durée du produit. Une réponse 200 confirme la commande et le frontend affiche les détails de la licence nouvellement activée.

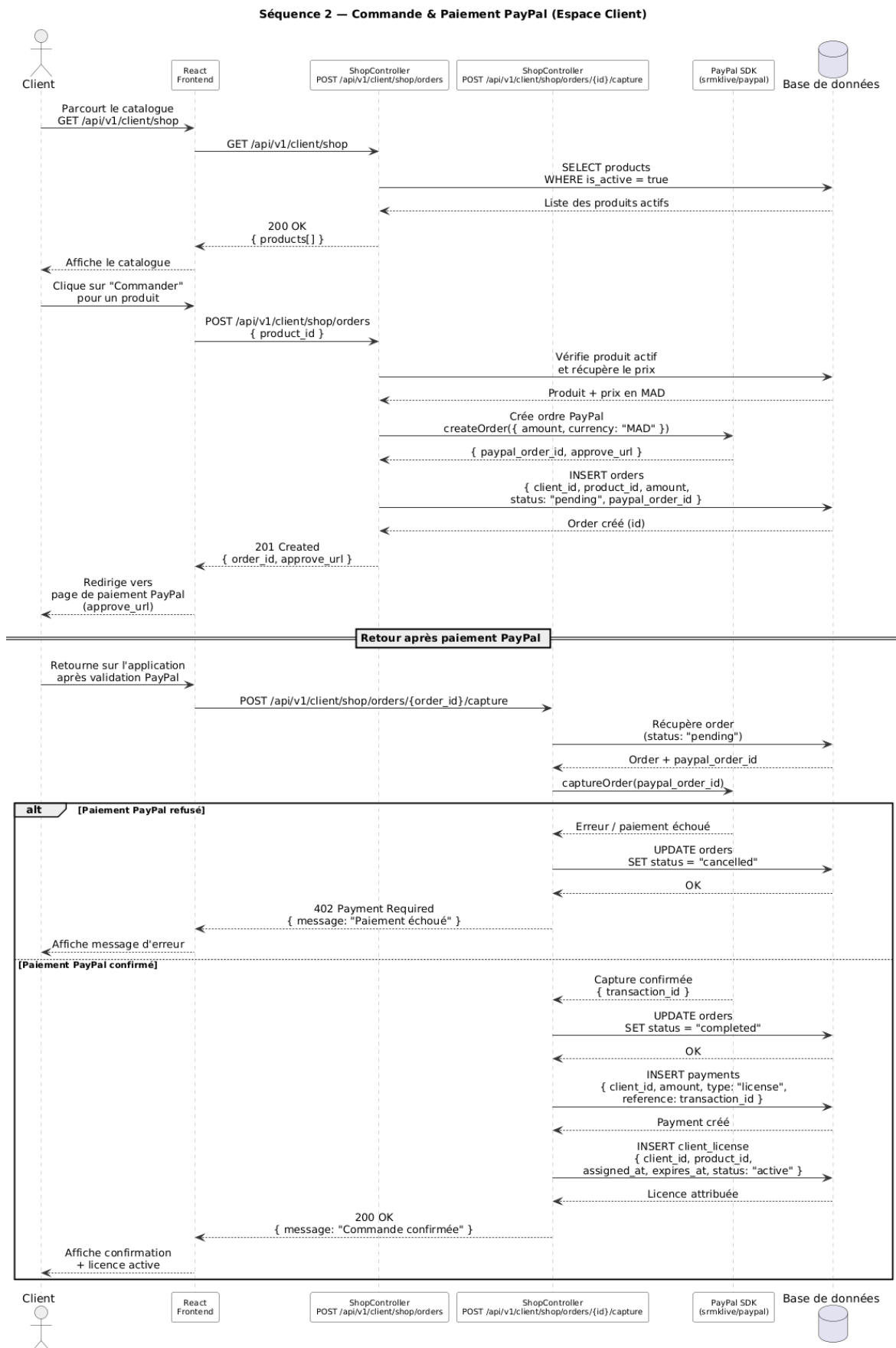


Figure 2.5: Diagramme de sequence – Commande et paiement PayPal

2.4.3 Sequence 3 – Verification automatique des licences

Le troisieme diagramme de sequence (Figure 2.6) illustre le fonctionnement du systeme d'automatisation base sur le planificateur de taches Laravel. Ce scenario ne fait intervenir aucune action manuelle de la part d'un utilisateur : tout est declenche automatiquement.

Chaque jour a 08h00, le Scheduler dispatch le job `CheckLicenseExpiry`. Ce job interroge la base pour recuperer toutes les licences dont la date d'expiration est inferieure ou egale a la date du jour. Pour chacune d'elles, il met a jour le statut a `expired`. Il effectue ensuite une seconde requete pour identifier les licences dont l'expiration se situe entre maintenant et dans sept jours. Pour chacune, il met le statut a `expiring_soon` et dispatch la notification `LicenseExpiringNotification`. Cette notification est envoyee via deux canaux en parallele : le canal database, qui insere une entree dans la table `notifications` consultable depuis l'espace client, et le canal mail, qui envoie un email via Laravel Mailer avec un lien vers la boutique pour renouveler la licence.

Toujours a 08h00, le Scheduler dispatch le job `SendLicenseExpiryReminder`. Celui-ci interroge la base pour trouver les licences dont la date d'expiration correspond exactement a aujourd'hui plus sept jours et dont le statut est `expiring_soon`. Pour chaque client concerne, il envoie individuellement un email de rappel avec un lien vers la boutique.

Enfin, quand le client consulte ses notifications via `GET /api/v1/client/notifications`, il recoit la liste des notifications lues et non lues. Il peut ensuite marquer une notification comme lue via `PATCH /api/v1/client/notifications/{id}/read`, ce qui retourne un 200 confirmant la mise a jour.

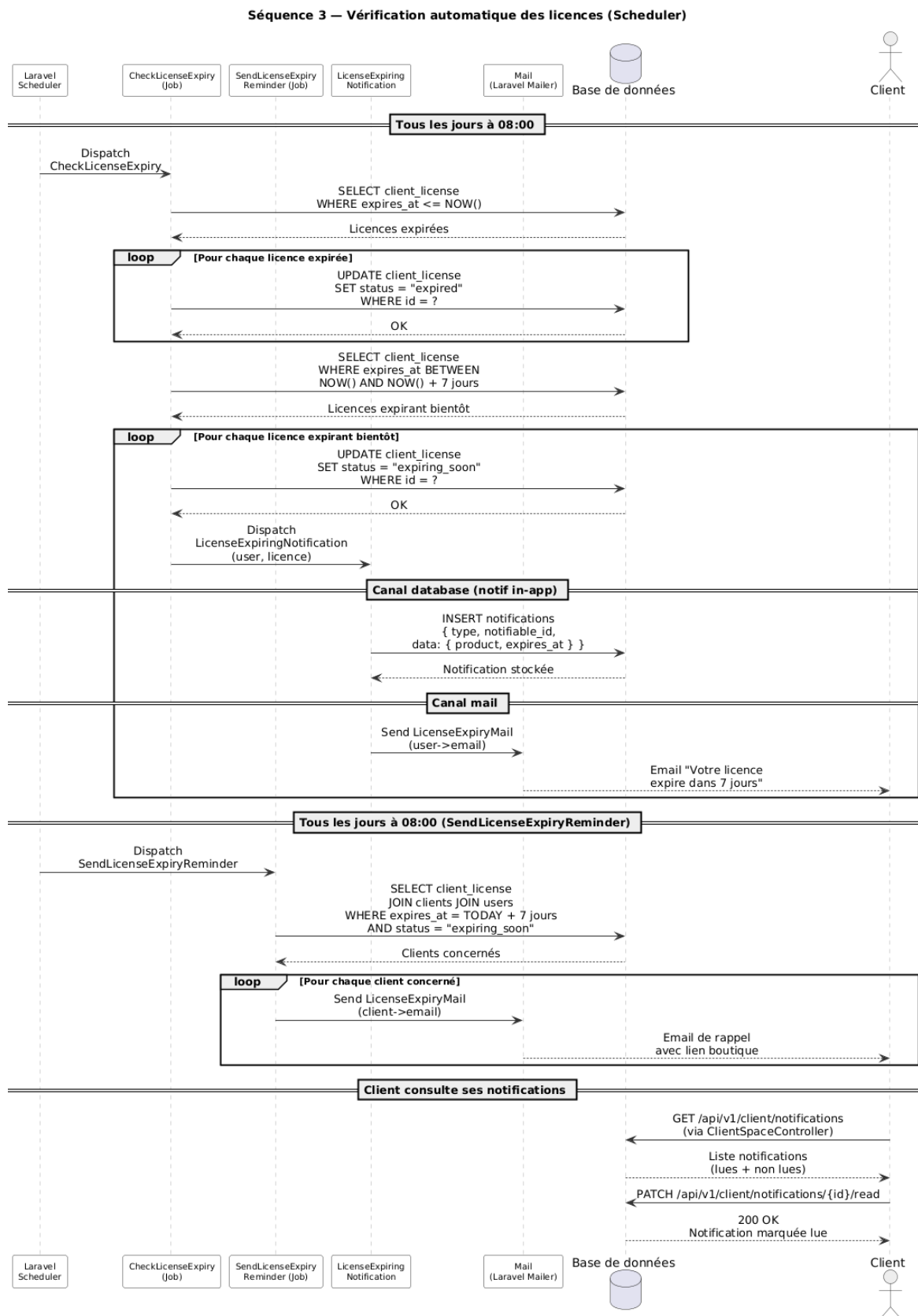
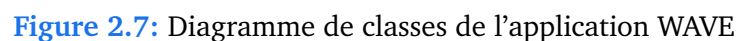


Figure 2.6: Diagramme de sequence – Verification automatique des licences

2.5 Diagramme de classes

Le diagramme de classes (Figure 2.7) représente la structure statique du système. Il décrit les entités du domaine, leurs attributs, leurs méthodes et les relations qui les unissent. Il constitue la référence directe pour la conception du schéma de base de données.



La classe **User** represente tout utilisateur authentifie dans le systeme, quel que soit son role. Elle stocke les informations d'identite (nom, email, mot de passe hache) et le champ `role` qui determine les droits d'accès. Un User peut avoir une relation HasOne vers un Client (dans le cas d'un utilisateur client provisionne par le manager),

et des relations HasMany vers les entites ManagerGoal, FollowUp, Contract (en tant que manager responsable) et Opportunity.

La classe **Client** est la pierre angulaire de la logique metier. Elle contient les informations commerciales du client (nom, email, telephone, societe, statut, notes) et le champ `deleted_at` qui permet les suppressions douces. Un Client est relie a un User via une cle etrangere `user_id` nullable (nullable car un client peut exister sans compte utilisateur). Il entretient des relations HasMany avec les classes Payment, Invoice, Contract, FollowUp, Order et SupportTicket. La relation avec Product est de type ManyToMany via la table pivot `client_license`, qui stocke les dates d'attribution et d'expiration ainsi que le statut de chaque licence.

La classe **Product** stocke les informations du catalogue : nom, description, prix en MAD, duree en jours et le flag `is_active` qui controle la visibilite en boutique. Elle est reliee a Client via la table pivot `client_license`, et a Contract et Order par des relations BelongsTo.

La classe **Payment** enregistre chaque transaction financiere avec son montant, sa date, sa reference et son type via un enum (`subscription`, `license`, `other`). Elle supporte les suppressions douces. Elle a une relation HasOne vers Invoice, ce qui lie automatiquement une facture a son paiement d'origine.

La classe **Invoice** stocke les factures avec leur numero au format FAC-YYYY-NNNN, leur statut, leurs dates d'emission et d'echeance. Elle supporte les suppressions douces et est reliee a Client et Payment via des cles etrangeres.

La classe **Contract** modelise les engagements contractuels entre l'agence et ses clients. Elle reference a la fois le client, le produit concerne et le manager responsable. Un scope Eloquent `expiringWithin` est defini pour filtrer facilement les contrats expirant dans un delai donne.

Les classes **Opportunity** et **FollowUp** couvrent respectivement le pipeline commercial CRM et les relances commerciales. Toutes deux referencent un client et un utilisateur manager. FollowUp dispose d'un champ booleen `is_done` pour marquer une relance comme effectuee.

La classe **Order** enregistre les commandes passees via PayPal, avec le champ `paypal_order_id` permettant de faire le lien avec la transaction PayPal et un enum de statut (`pending`, `completed`, `cancelled`).

La classe **SupportTicket** modelise les tickets de support soumis par les clients, avec leur sujet, leur message et leur statut.

Enfin, la classe **ManagerGoal** stocke les objectifs mensuels definis par l'admin pour chaque manager, avec le mois, l'annee et le montant cible en MAD.

Le tableau 2.1 synthetise les principales relations entre les classes du systeme.

Table 2.1: Principales relations Eloquent du systeme

Classe source	Type	Classe cible
User	HasOne	Client
User	HasMany	ManagerGoal, FollowUp, Contract (manager), Opportunity
Client	BelongsTo	User
Client	HasMany	Payment, Invoice, Contract, FollowUp, Order, SupportTicket
Client	BelongsToMany	Product (via client_license)
Payment	HasOne	Invoice
Invoice	BelongsTo	Client, Payment
Contract	BelongsTo	Client, Product, User (manager)
Opportunity	BelongsTo	Client, User (manager)
Order	BelongsTo	Client, Product
SupportTicket	BelongsTo	Client
ManagerGoal	BelongsTo	User (manager)

2.6 Architecture technique

2.6.1 Architecture API-first

L'application repose sur une architecture API-first avec une separation nette entre le backend et le frontend. Le backend est un serveur Laravel qui expose une API REST sous le prefixe `/api/v1`. Le frontend est une Single Page Application React qui consomme cette API via des requetes HTTP authentifiees. Les deux serveurs fonctionnent de maniere independante et communiquent exclusivement par echanges JSON.

Cette separation offre plusieurs avantages concrets : le frontend peut etre deploye sur un hebergeur statique sans dependre de l'infrastructure PHP, le backend peut etre consomme par d'autres clients (application mobile, outil tiers) sans modification, et les deux couches peuvent evoluer independamment l'une de l'autre.

2.6.2 Gestion des roles et des acces

Le controle d'accès est implemente a deux niveaux. Cote backend, le middleware `RoleMiddleware` verifie le champ `role` de l'utilisateur authentifie avant d'au-

toriser l'accès à chaque groupe de routes. Les routes admin sont protégées par `middleware(['auth:sanctum', 'role:admin'])`, les routes back-office par `role:admin,manager`, et les routes espace client par `role:client`.

Côté frontend, le composant `PrivateRoute` inspecte le rôle stocké dans le contexte d'authentification et redirige automatiquement tout utilisateur qui tenterait d'accéder à une section qui ne lui appartient pas.

2.6.3 Gestion asynchrone et tâches planifiées

Les opérations longues ou susceptibles d'échouer, notamment l'envoi d'emails, sont traitées de manière asynchrone via le système de queues de Laravel. Cela garantit que les actions de l'utilisateur reçoivent une réponse immédiate sans attendre la fin de traitements externes.

Les tâches récurrentes sont définies dans `routes/console.php` et orchestrées par le scheduler Laravel, qui agit comme un cron unifié. Les trois tâches planifiées sont la vérification quotidienne des licences à 08h00, l'envoi des rappels de retard de paiement chaque lundi à 09h00 et la génération du rapport mensuel le premier de chaque mois à 07h00.

2.7 Conclusion

Ce deuxième chapitre a posé les fondations conceptuelles et architecturales de l'application WAVE. Les diagrammes de cas d'utilisation ont permis de couvrir exhaustivement l'ensemble des interactions entre les quatre acteurs du système et la plateforme. Les diagrammes de séquence ont mis en lumière les flux les plus complexes : la sécurisation de l'authentification par token Bearer avec redirection par rôle, le processus de commande et de paiement PayPal avec attribution automatique de licence, et le système d'automatisation par tâches planifiées pour le suivi des licences.

Le diagramme de classes a révélé une architecture de données cohérente, organisée autour des entités `Client` et `User`, avec des relations Eloquent bien définies qui facilitent les requêtes et garantissent l'intégrité référentielle des données.

Ces éléments constituent la référence directe qui a guidé le développement technique. Le chapitre suivant présentera la réalisation de l'application : les choix technologiques justifiés, les modules implémentés et les interfaces produites.